



BITS Pilani
DIGITAL
Excellence made yours

Pipeline Orchestration Basics

From manual scripts to automated,
dependency-driven ML workflows

- Topic 1: Workflow Automation Fundamentals
- Topic 2: Deep Dive into Apache Airflow
- Topic 3: Designing an ETL-to-Training Pipeline
- Topic 4: Hands-on Tooling Setup
- Topic 5: Lesson Summary

Lesson Overview



BITS Pilani
DIGITAL
Excellence made yours

Lesson 2 · Pipeline Orchestration Basics

5

Topics

Core topic areas

16

Concept Blocks

Short explainers

~65

Duration

Total minutes

FON

Level

Foundational

LEARNING OBJECTIVES

1 Understand the fundamentals of workflow automation and why it matters for ML

2 Master core concepts: DAGs, tasks, dependencies, and scheduling

3 Build and run a basic ETL-to-training pipeline using Apache Airflow



TOPIC 0

Lesson Introduction

What to Expect This Week

What to Expect This Week

Four topic areas, hands-on setup, and a full recap

Core Concepts

DAGs, tasks, dependencies, and scheduling — the vocabulary of pipeline orchestration

Apache Airflow Deep Dive

Architecture: Scheduler, Executor, Web Server, Metadata DB, operators and sensors

Pipeline Design

Connect Extract → Validate → Transform → Train into a production-ready ETL pipeline

Hands-on Tooling Setup

Install and configure Apache Airflow locally to run your first DAG



TOPIC 1

Workflow Automation Fundamentals for AI

Why manual scripts break at scale — and what DAGs do instead

Why Workflow Automation Matters in AI/ML



The hidden cost of manually chaining ML scripts

Without Orchestration

- Scripts run manually, one by one
- No visibility into what failed and why
- Re-run entire pipeline even if one step failed
- No scheduling — someone must remember to trigger it
- Impossible to scale across models or environments

With Orchestration

- Define tasks once — scheduler runs automatically
- Full lineage: see exactly which task failed and retry it
- Only failed tasks re-run — not the whole pipeline
- Schedule by cron, event, or sensor trigger
- Same DAG runs across dev, staging, and production

DAGs, Tasks, and Dependencies — Core Concepts



The three building blocks every orchestration tool shares

DAG

Directed Acyclic Graph

A graph of tasks where arrows flow in one direction with no loops. The blueprint of your pipeline.

Task

Unit of Work

A single step in the pipeline — e.g. `extract_data`, `train_model`. Each task is a Python function.

Edge

Dependency

An arrow from Task A to Task B meaning B runs only after A succeeds. Defines execution order.

DAG Run

Pipeline Execution Instance

One complete execution of the DAG. Multiple runs can execute in parallel.

Branching, Parallelism, and Conditional Execution



Real pipelines don't always follow a straight line

Branching

A task examines a condition and routes execution to one of several downstream paths.

Example: `accuracy < 80%` → `retrain_task`. Otherwise → `deploy_task`.

Airflow: `BranchPythonOperator`

Parallelism

Independent tasks run simultaneously across multiple workers.

Example: `validate_features` and `validate_schema` run in parallel — reducing total pipeline time.

Conditional Skip

A task is skipped if upstream conditions are not met, without failing the whole pipeline.

Example: Skip retraining if new data volume is too small.

Airflow: `ShortCircuitOperator`

Scheduling Strategies — Cron, Event-Driven, and Hybrid



When and how pipelines are triggered



Cron-Based

Example: `schedule_interval = "0 6 * * *"`

Use when: Fixed-time: daily retraining at 6 AM, weekly batch reports on Sunday.



Simple, predictable, no trigger logic needed



Event-Driven

Example: S3 ObjectCreated trigger, Kafka message

Use when: React to new data arriving — run pipeline when a new training batch lands in S3.



Near real-time, eliminates unnecessary empty runs



Hybrid

Example: Cron + S3KeySensor

Use when: Schedule a daily run but pause until an upstream data dependency sensor succeeds.



Flexible and production-safe — best of both approaches

Orchestration Tool Landscape — Airflow, Prefect, and Dagster

How the three leading tools compare for ML workloads



BITS Pilani
DIGITAL
Excellence made yours

Apache Airflow

Industry Standard

✓ Strengths

- Huge ecosystem & community
- Python-native DAGs
- Battle-tested at scale
- Rich UI, logging, alerts

⚠ Considerations

- Complex setup
- Not natively dynamic
- Scheduler as single point of failure

Prefect

Modern & Pythonic

✓ Strengths

- Dynamic workflows
- Simple decorator API
- Hybrid cloud support
- Good error observability

⚠ Considerations

- Smaller community
- Best DX needs managed cloud
- Newer, less enterprise adoption

Dagster

Asset-Oriented

✓ Strengths

- Software-defined assets
- Built-in data lineage
- Type-safe I/O
- Strong testing tools

⚠ Considerations

- Steeper learning curve
- Asset model requires rethinking
- Smaller talent pool



TOPIC 2

Deep Dive into Apache Airflow

Architecture, components, and patterns that power production pipelines

Airflow Architecture — Scheduler, Executor, Web Server, Metadata DB

Four components that work together to run your DAGs



BITS Pilani
DIGITAL
Excellence made yours



Scheduler

Parses DAG files, triggers task instances when dependencies are met and schedule is due.



Executor

Runs the actual task. LocalExecutor (single machine), CeleryExecutor (distributed), KubernetesExecutor (K8s pods).



Web Server

Flask-based UI for monitoring DAG runs, viewing logs, triggering manually, and managing connections.



Metadata DB

PostgreSQL stores all DAG/task state, run history, XComs, variables, and connections.



DAG Files

Python files defining your workflows. The Scheduler reads these from /dags on a polling interval.



Worker

Separate processes that execute task code. Number of workers determines pipeline parallelism.

Anatomy of an Airflow DAG — Operators, Sensors, and Hooks

The building blocks inside every DAG file



BITS Pilani
DIGITAL
Excellence made yours



Operators (What runs)

- PythonOperator — runs a Python function
- BashOperator — runs a shell command
- BranchPythonOperator — routes to one of multiple downstream tasks
- EmailOperator — sends alerts on failure
- SageMakerTrainingOperator — launches AWS training job
- SparkSubmitOperator — submits a Spark job



Sensors & Hooks

Sensors — wait for a condition before proceeding

- S3KeySensor waits for a file to appear in S3
- ExternalTaskSensor waits for another DAG to succeed

Hooks — reusable connectors to external systems

- PostgresHook, S3Hook, HttpHook

Connections — stored credentials for hooks, managed in Airflow UI or AWS Secrets Manager

Airflow Variables, Connections, and XComs for Data Passing

How tasks store config and share data with each other



BITS Pilani
DIGITAL
Excellence made yours

Variables

Key-value pairs stored in the Metadata DB. Use for global config.

Example:
airflow variables set
MODEL_THRESHOLD 0.80

```
from airflow.models import Variable
threshold =
Variable.get("MODEL_THRESHOLD")
```

Connections

Named credentials for external systems.

Example:
aws_default — AWS credentials
postgres_default — DB connection

Reference in hooks:
hook =
S3Hook(aws_conn_id="aws_default")

XComs (Cross-Communication)

Share small values between tasks downstream.

Push: ti.xcom_push(key='accuracy',
value=0.87)
Pull: acc =
ti.xcom_pull(task_ids='evaluate',
key='accuracy')

 XComs stored in Metadata DB —
keep values small. Use S3 for large
DataFrames.

Airflow Best Practices — Error Handling, Retries, and Logging



BITS Pilani
DIGITAL
Excellence made yours

Six rules that separate a working DAG from a production DAG



Retries

Set retries and `retry_delay` in `default_args`. Start with `retries=2`, `delay=5min`. Exponential backoff for external APIs.



Idempotent Tasks

Each task produces the same result if run multiple times. Never append to files — always overwrite. Use partition keys or timestamps.



Atomic Tasks

Tasks should be small and focused — one logical action per task. Avoid side effects. Easier to retry and debug.



Structured Logs

Use Python logging, not `print()`. Logs are captured per task in the Airflow UI. Include `task_id`, `run_id`, and record counts.



Timeouts

Set `execution_timeout` on each task. Long-running tasks must have a maximum allowed duration to prevent blocking workers.



Alerting

Use `on_failure_callback` to trigger PagerDuty or Slack. Use SLA miss callbacks for pipeline latency alerting.

Monitoring Airflow DAGs — Logs, Metrics, and UI Overview

How to know your pipeline is healthy in production



BITS Pilani
DIGITAL
Excellence made yours



Airflow UI — What to Watch

- Grid View — timeline of all past runs with colour status (green=success, red=failed, yellow=running)
- Task Logs — per-task stdout/stderr captured automatically
- Gantt Chart — visualise where time is spent across tasks
- Task Duration graph — track performance regressions
- SLA Misses panel — tasks that exceeded their deadline



External Metrics (StatsD + Grafana)

- Airflow emits StatsD metrics for scheduler heartbeat, task success/fail rate, executor queue depth
- Feed into Prometheus via statsd-exporter and visualise in Grafana
- Key metric: scheduler.tasks.running — should match concurrency config
- Alert on: dag_processing.total_parse_time > 30s (DAG file becoming too large)



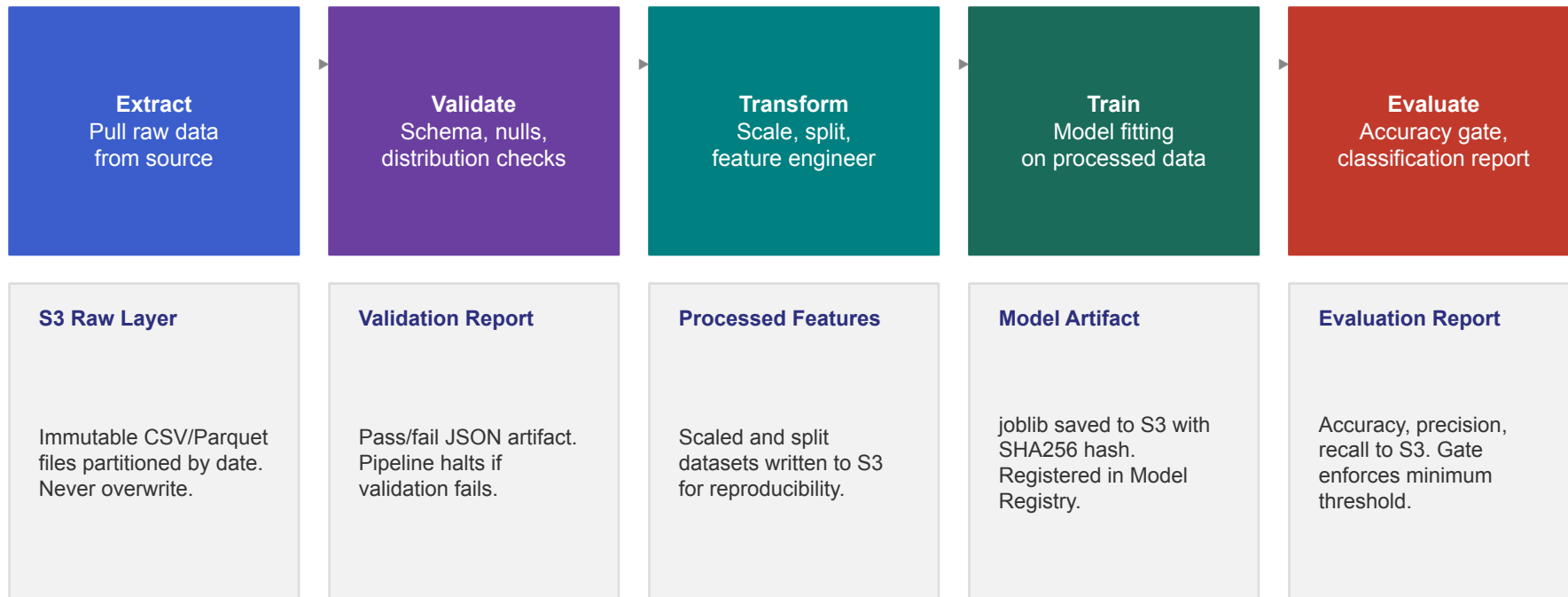
TOPIC 3

Designing an ETL-to-Training Pipeline

Connecting data ingestion, validation, transformation, and model training

Pipeline Design — From Data Ingestion to Model Training

Five stages with clear inputs, outputs, and S3 artifact handoffs



Connecting Pipeline Stages — Validation, Transformation, Training

How S3 artifact paths and XComs wire pipeline stages together



BITS Pilani
DIGITAL
Excellence made yours

How Stages Connect

- Each stage writes output to S3 and stores the path in XCom
- The next stage reads the path from XCom — never hardcode paths in task code
- Validate fails → Transform is skipped → Train is skipped
- Each stage is independently retryable
- Parameterise inputs: run_date, data_version so the same DAG handles multiple time windows

S3 Output per Stage

- Extract → s3://bucket/data/raw/<date>/loan.csv
- Validate → s3://bucket/data/validated/<date>/report.json
- Transform → s3://bucket/data/processed/<date>/train.csv
s3://bucket/data/processed/<date>/test.csv
- Train → s3://bucket/models/<run_id>/model.joblib
- Evaluate → s3://bucket/evaluations/<run_id>/report.json

Data Validation Steps Within the Pipeline

Six checks that protect every downstream stage from bad data

Schema Check

Verify all expected columns exist with the right data types. Reject if schema has changed upstream.

Null Check

Count null values per column. Fail if any required column exceeds a null threshold (e.g. > 5%).

Row Count

Ensure the extract produced a minimum number of rows. Empty extracts indicate upstream problems.

Distribution Check

Compare feature distributions against a baseline (KS test, PSI). Flag if significant drift detected.

Target Balance

Check class imbalance ratio. Severe imbalance (99:1) without handling will produce misleading models.

Duplicate Check

Remove or count duplicates. Duplicates in training data cause overfitting and inflate accuracy metrics.



TOPIC 4

04 Hands-on Tooling Setup

Installing and configuring Apache Airflow locally

Installing and Configuring Apache Airflow Locally



Four steps from zero to your first running DAG

1 Install Airflow (Python 3.9+)

```
pip install apache-airflow==2.9.1 \
--constraint https://raw.githubusercontent.com/apache/
airflow/constraints-2.9.1/constraints-3.10.txt
```

2 Initialise the Database

```
export AIRFLOW_HOME=~/.airflow
airflow db init
```

3 Create Admin User

```
airflow users create \
--username admin --password admin \
--firstname Dev --lastname User \
--role Admin --email admin@example.com
```

4 Start Scheduler and Web Server

```
# Terminal 1
airflow scheduler

# Terminal 2
airflow webserver --port 8080
```

✓ **Access Airflow UI at: <http://localhost:8080> | Login: admin / admin**



TOPIC 5

05 Lesson Summary

Week 2 recap and key takeaways

Week 2 Recap — Pipeline Orchestration Basics



Four things to take away from this lesson

1

DAGs Define Order

A DAG is a directed, acyclic graph of tasks. Tasks only run when all upstream dependencies succeed — making pipelines safe and reproducible.

2

Airflow = Scheduler + Executor + UI + DB

Four components work together: Scheduler decides what to run, Executor runs it, Web Server shows you the state, Metadata DB stores everything.

3

Data Flows via XComs and S3

Small values (paths, counts, accuracy) move via XComs. Large data (DataFrames, models) must go through S3 — never put large data in XComs.

4

Validation Gates Protect Production

Add a validation task after every data ingestion and a quality gate after training. Stop the pipeline early to avoid serving a bad model.

Advanced Pipeline Orchestration

Kubeflow Pipelines · Ray Distributed · MLflow · Automated Retraining · Drift Detection Triggers



Kubeflow Pipelines

Kubernetes-native ML workflow orchestration



Ray for Distributed

Parallel and distributed training at scale



MLflow Integration

Experiment tracking and model registry



Auto-Retraining

Trigger retraining on drift or schedule